SCHOOL OF COMPUTER APPLICATION

BBD UNIVERSITY

SUBMITTED BY

NAME- Vaibhavi vishwakarma

SECTION- BCA(DS-AI)-24

ROLL NO.- 1240258482

TO Vikas sir

BABU BANARSI DAS UNIVERSITY, LUCKNOW

PROJECT

Sales Dataset MapReduce Practical

Architecture:

• The architecture follows a standard Big Data pipeline using Hadoop ecosystem components.

Data Flow:

• Raw Dataset HDFS MapReduce Processing Processed Data Hive Analysis Insights

Explanation:

• Raw sales dataset (CSV) is first stored in HDFS

• MapReduce is used to process the data and calculate total sales per category

• Processed data is stored back in HDFS

• Hive is used to perform analytical queries on the results

• Final insights such as top-selling categories are generated

Tools Used:

• Hadoop Distributed File System (HDFS)

• MapReduce (Hadoop Streaming using Python)

• Hive (for data analysis)

• Linux Terminal

• ChatGPT (Generative AI tool)

Technologies:

• Python (for mapper and reducer)

• CSV dataset

• Hadoop Ecosystem

Use of Generative AI in the Project:

• In this project, Generative AI tools were used to assist in development and understanding of Big Data concepts.

Areas Where AI Was Used:

1. Code Generation
   • Generative AI was used to generate MapReduce programs (mapper and reducer).
2. Debugging
   • Errors in code were identified and fixed using AI assistance.
3. Query Writing

- Hive queries were generated and optimized using AI.

4. Concept Understanding
   - AI helped in understanding HDFS architecture, MapReduce workflow, and Hive operations.

Example Prompt Used:

- Write a Python MapReduce program to calculate total sales per product category.

Benefits of Using AI:

- Faster development
- Reduced coding errors
- Better understanding of concepts
- Improved efficiency

Validation:

- All AI-generated code was manually tested and verified before execution.

Dataset Description:

- The dataset used is a sales dataset containing product sales information across different categories and regions. The data is stored in CSV (Comma Separated Values) format.

Dataset Structure:

- The dataset consists of the following columns:
  - order_id – Unique ID of the order
  - product – Name of the product
  - category – Product category (Electronics, Furniture, etc.)
  - price – Price of one unit
  - quantity – Number of units sold
  - region – Region where the sale occurred
  - date – Date of sale
  - customer_id – Unique ID of the customer
  - payment_mode – Mode of payment (Cash/Card/UPI)
  - discount – Discount applied (%)

Dataset Source:

- The dataset is a combination of real-world sales data and custom entries created for project demonstration.

Purpose of Dataset:

- The dataset is used to:
  - Analyze total sales per product category
  - Find top-selling categories
  - Perform data aggregation using MapReduce
  - Generate insights using Hive

Sample Data:

```
order_id,product,category,price,quantity,region,date,customer_id,payment_mode,discount
1,Laptop,Electronics,55000,2,North,2024-01-10,C001,Card,5
2,Chair,Furniture,4500,5,South,2024-01-11,C002,UPI,0
3,Phone,Electronics,20000,3,East,2024-01-12,C003,Cash,10
4,Desk,Furniture,8000,2,West,2024-01-13,C004,Card,0
5,Laptop,Electronics,55000,1,South,2024-01-14,C005,UPI,5
6,Headphones,Electronics,3000,4,North,2024-01-15,C006,Card,0
7,Chair,Furniture,4500,3,East,2024-01-16,C007,Cash,0
8,Tablet,Electronics,25000,2,West,2024-01-17,C008,UPI,8
9,Sofa,Furniture,15000,1,North,2024-01-18,C009,Card,0
10,Phone,Electronics,20000,2,South,2024-01-19,C010,Cash,10
```

Conclusion:

• In this project, a complete Big Data pipeline was successfully implemented using Hadoop ecosystem components.

• The dataset was stored in HDFS, processed using MapReduce to calculate total sales per category, and analyzed using Hive to extract meaningful insights such as top-selling categories.

• The project helped in understanding the working of distributed data processing and data analysis techniques.

• Generative AI acted as a smart assistant throughout the project by helping in code generation, debugging, and understanding complex concepts.

O Generative AI was used not only for code generation but also as a learning assistant to understand Big Data workflows and improve overall efficiency.

Question-1:- Sales Data Analysis using Hadoop Streaming.

O Aim: - "To analyze sales dataset and calculate total sales per category using

Hadoop MapReduce."

Step-01:- Create a "CSV" file.

```
I cat > sales.csv

[cloudera@quickstart ~]$ cat > sales.csv
order_id,product,category,price,quantity,region,date,customer_id,payment_mode,discount
1,Laptop,Electronics,55000,2,North,2024-01-10,C001,Card,5
2,Chair,Furniture,4500,5,South,2024-01-11,C002,UPI,0
3,Phone,Electronics,20000,3,East,2024-01-12,C003,Cash,10
4,Desk,Furniture,8000,2,West,2024-01-13,C004,Card,0
5,Laptop,Electronics,55000,1,South,2024-01-14,C005,UPI,5
6,Headphones,Electronics,3000,4,North,2024-01-15,C006,Card,0
7,Chair,Furniture,4500,3,East,2024-01-16,C007,Cash,0
8,Tablet,Electronics,25000,2,West,2024-01-17,C008,UPI,8
9,Sofa,Furniture,15000,1,North,2024-01-18,C009,Card,0
10,Phone,Electronics,20000,2,South,2024-01-19,C010,Cash,10
11,Laptop,Electronics,55000,3,East,2024-01-20,C011,Card,5
12,Chair,Furniture,4500,4,West,2024-01-21,C012,Cash,0
13,Headphones,Electronics,3000,6,North,2024-01-22,C013,UPI,0
14,Sofa,Furniture,15000,2,South,2024-01-23,C014,Card,0
15,Tablet,Electronics,25000,1,East,2024-01-24,C015,Cash,8
16,Desk,Furniture,8000,3,West,2024-01-25,C016,UPI,0
17,Phone,Electronics,20000,4,North,2024-01-26,C017,Card,10
18,Laptop,Electronics,55000,2,South,2024-01-27,C018,Cash,5
19,Chair,Furniture,4500,6,East,2024-01-28,C019,UPI,0
20,Tablet,Electronics,25000,3,West,2024-01-29,C020,Card,8
[cloudera@quickstart ~]$
```

Step-02:- Create a directory

I hdfs dfs -mkdir /user/cloudera/vaibhavi

I hdfs dfs -mkdir /user/cloudera/vaibhavi/sales

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/vaibhavi
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/vaibhavi/sales
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/vaibhavi
Found 1 items
drwxr-xr-x   - cloudera cloudera          0 2026-04-20 08:39 /user/cloudera/vaibhavi/sales
[cloudera@quickstart ~]$
```

Step-03:- Load the "CSV" on Cloudera's Server

I hdfs dfs -put sales.csv /user/cloudera/vaibhavi/sales

I hdfs dfs -ls /user/cloudera/vaibhavi/sales

```
[cloudera@quickstart ~]$ hdfs dfs -put sales.csv /user/cloudera/vaibhavi/sales
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/vaibhavi/sales
Found 1 items
-rw-r--r--   1 cloudera cloudera       1623 2026-04-20 08:41 /user/cloudera/vaibhavi/sales/sales.csv
[cloudera@quickstart ~]$
```

Step-04:- Create a mapper file using "nano"

l nano mapper.py

- #!/usr/bin/env python
- import sys
- for line in sys.stdin:
- if "order_id" in line:
- continue
- data = line.strip().split(',')
- category = data[2]
- total = float(data[3]) * int(data[4])
- print("%s\t%s" % (category, total))

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
        if "order_id" in line:
                continue
        data = line.strip().split(',')
        category = data[2]
        total = float(data[3]) * int(data[4])
        print("%s\t%s" % (category, total))
```

Step-05:- Create a reducer file using "nano"

```
I nano reducer.py
• #!/usr/bin/env python
• import sys
•
• current_category = None
• total_sales = 0
•
• for line in sys.stdin:
• category, sales = line.strip().split('\t')
• sales = float(sales)
•
• if current_category == category:
• total_sales += sales
• else:
• if current_category:
• print("%s\t%.2f" % (current_category, total_sales))
• current_category = category
• total_sales = sales
•
• if current_category:
• print("%s\t%.2f" % (current_category, total_sales))
```

```
#!/usr/bin/env python
import sys

current_category = None
total_sales = 0

for line in sys.stdin:
	category, sales = line.strip().split('\t')
	sales = float(sales)

	if current_category == category:
		total_sales += sales
	else:
		if current_category:
			print("%s\t%.2f" % (current_category, total_sales))
		current_category = category
		total_sales = sales

if current_category:
	print("%s\t%.2f" % (current_category, total_sales))
```

Step-06:- Make executable

I chmod +x mapper.py reducer.py

```
[cloudera@quickstart mapreduce]$ chmod +x mapper.py reducer.py
[cloudera@quickstart mapreduce]$ ls
mapper.py                                reducer.py
[cloudera@quickstart mapreduce]$ ls -l
total 8
-rwxrwxr-x 1 cloudera cloudera 198 Apr 20 08:46 mapper.py
-rwxrwxr-x 1 cloudera cloudera 431 Apr 20 08:49 reducer.py
[cloudera@quickstart mapreduce]$
```

Step-07:- Copy "sales.csv" file from /home/cloudera to

/home/cloudera/mapreduce

I cp /home/cloudera/sales.csv

```
/home/cloudera/mapreduce/sales.csv

[cloudera@quickstart mapreduce]$ cp /home/cloudera/sales.csv /home/cloudera/mapreduce/sales.csv
[cloudera@quickstart mapreduce]$ ls
mapper.py    sales.csv    reducer.py
[cloudera@quickstart mapreduce]$
```

Step-08:- Local test

I cat sales.csv | python mapper.py

I cat sales.csv | python mapper.py | sort | python reducer.py

```
[cloudera@quickstart mapreduce]$ cat sales.csv | python mapper.py
Electronics                    110000.0
Furniture                      22500.0
Electronics                    60000.0
Furniture                      16000.0
Electronics                    55000.0
Electronics                    12000.0
Furniture                      13500.0
Electronics                    50000.0
Furniture                      15000.0
Electronics                    40000.0
[cloudera@quickstart mapreduce]$ cat sales.csv | python mapper.py | sort | python reducer.py
Electronics                    627000.00
Furniture                      175500.00
```

Step-09:- Run Hadoop Job

I hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar

-files mapper.py,reducer.py -input

/user/cloudera/vaibhavi/sales/sales.csv -output

/user/cloudera/vaibhavi/sales/output -mapper "python

mapper.py" -reducer "python reducer.py"

```
[cloudera@quickstart mapreduce]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
    -files mapper.py,reducer.py \
    -input /user/cloudera/vaibhavi/sales/sales.csv \
    -output /user/cloudera/vaibhavi/sales/output \
    -mapper "python mapper.py" -reducer "python reducer.py"
packageJobJar: [] [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] tmpDir=null
26/04/20 08:58:31 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/20 08:58:33 INFO mapreduce.JobSubmitter: number of splits: 2
26/04/20 08:58:35 INFO mapreduce.Job: Running job: job_1776698867558_0001
26/04/20 08:58:51 INFO mapreduce.Job: map 0% reduce 0%
26/04/20 08:59:02 INFO mapreduce.Job: map 50% reduce 0%
26/04/20 08:59:03 INFO mapreduce.Job: map 100% reduce 0%
26/04/20 08:59:15 INFO mapreduce.Job: map 100% reduce 100%
26/04/20 08:59:16 INFO mapreduce.Job: Job job_1776698867558_0001 completed successfully
26/04/20 08:59:16 INFO mapreduce.Job: Counters: 49
```

Step-010:- View Output

I hdfs dfs -cat

/user/cloudera/vaibhavi/sales/output/part-00000

```
[cloudera@quickstart mapreduce]$ hdfs dfs -ls /user/cloudera/vaibhavi/sales/output
Found 2 items
    -rw-r--r--   1 cloudera cloudera          0 2026-04-20 08:59 .../output/_SUCCESS
    -rw-r--r--   1 cloudera cloudera        219 2026-04-20 08:59 .../output/part-00000
[cloudera@quickstart mapreduce]$ hdfs dfs -cat /user/cloudera/vaibhavi/sales/output/part-00000
Electronics                    627000.00
Furniture                      175500.00
[cloudera@quickstart mapreduce]$
```

Question-2:- To perform data analysis on processed sales dataset using Hive.

O Aim: - "To analyze sales data and find top-selling categories using Hive queries."

Step-01:- Open Hive

I hive

```
[cloudera@quickstart mapreduce]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

Step-02:- Create table on hive

I CREATE TABLE sales_total (

I category STRING,

I total_sales FLOAT

I)

I ROW FORMAT DELIMITED

I FIELDS TERMINATED BY '\t';

```
hive> CREATE TABLE sales_total (
    > category STRING,
    > total_sales FLOAT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t';
OK
Time taken: 4.05 seconds
hive> show tables;
OK
sales_total
Time taken: 0.382 seconds, Fetched: 1 row(s)
hive>
```

Step-03:- Load data on the table in sales_total

I LOAD DATA INPATH '/user/cloudera/vaibhavi/sales/output' INTO TABLE sales_total;

```
hive> LOAD DATA INPATH '/user/cloudera/vaibhavi/sales/output' INTO TABLE sales_total;
Loading data to table default.sales_total
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/sales_total/p
Table default.sales_total stats: [numFiles=1, totalSize=219]
OK
Time taken: 1.471 seconds
hive>
```

Step-04:- View Data

I SELECT * FROM sales_total;

```
hive> SELECT * FROM sales_total;
OK
Electronics                    627000.0
Furniture                      175500.0
Time taken: 0.857 seconds, Fetched: 2 row(s)
hive>
```

Step-05:- Find Top Category

I SELECT * FROM sales_total ORDER BY total_sales DESC;

```
hive> SELECT * FROM sales_total ORDER BY total_sales DESC;
Query ID = cloudera_20260420090909_04fba5c2-85cf-43ea-aea4-2b7756c7d776
Total jobs = 1
Launching Job 1 out of 1
2026-04-20 09:10:20,037 Stage-1 map = 100%,                    reduce = 0%, Cumulative CPU 3.05 sec
2026-04-20 09:10:31,713 Stage-1 map = 100%,                    reduce = 100%, Cumulative CPU 6.39 sec
MapReduce Total cumulative CPU time: 6 seconds 390 msec
OK
Electronics                    627000.0
Furniture                      175500.0
Time taken: 40.217 seconds, Fetched: 2 row(s)
hive>
```

Step-06:- Top Category by Sales

I SELECT * FROM sales_total ORDER BY total_sales DESC LIMIT 1;

```
hive> SELECT * FROM sales_total ORDER BY total_sales DESC LIMIT 1;
Query ID = cloudera_20260420091212_47c4e033-f57b-415a-9f23-cca2fe7d76d5
Total jobs = 1
Launching Job 1 out of 1
2026-04-20 09:13:05,391 Stage-1 map = 0%,  reduce = 0%
2026-04-20 09:13:14,846 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.05 sec
2026-04-20 09:13:26,308 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 5.01 sec
MapReduce Total cumulative CPU time: 5 seconds 10 msec
OK
Electronics  627000.0
Time taken: 35.365 seconds, Fetched: 1 row(s)
hive>
```

Output:

- Hive successfully displayed:
  - Total sales per category
  - Top-selling category (Electronics)

O In this practical, Hive was used to analyze processed sales data and retrieve meaningful insights using SQL-like queries.